## 1. Markov 决策过程 (MDP)

$(S, A, P, r)$

状态 $s$

动作 $a$

奖励 reward $r(s,a)$

转移概率

3元 $P(s'|s,a)$

4元 $P(s',r|s,a)$ $\quad P(s'|s,a) = \sum_{r\in R} P(s',r|s,a)$

策略 $\pi$

确定 $\pi: S \to A \quad s \mapsto \pi(s)$

随机 $\pi: S \to [0,1]^{|A|}$ $\quad \pi(a|s)$ 条件概率

回报 return $G = r + \gamma r' + \gamma^2 r'' + \cdots$ $\quad \gamma$ discount rate

$(s, a, s', a', \cdots) \mapsto (r, r', \cdots) \mapsto \mathbb{R}$ $\quad G$ 是一个 rv. 与 $\pi$ 有关

性质: $G = r + \gamma G'$

## 2. Bellman 方程

状态值函数 $v_\pi(s) = E_\pi(G|s)$ $\quad v_\pi: S \to \mathbb{R} \quad v: S \times \Pi \to \mathbb{R}$

动作值函数 $q_\pi(s,a) = E_\pi(G|s,a)$ $\quad q_\pi: S \times A \to \mathbb{R}$

[注] $G$ 是关于 $\pi$ 的 rv. 故在求条件期望时 $E_\pi(G|\cdot)$ 在展开时 $E_\pi(G|\times)$ 会一直留着 拆不开

$$v_\pi(s) = \sum_a \pi(a|s) q_\pi(s,a)$$

$$v_\pi(s) = E_\pi(G|s) = \sum_a \pi(a|s) E_\pi(G|s,a) = \sum_a \pi(a|s) q_\pi(s,a)$$

$$q_\pi(s,a) = \sum_{s',r} P(s',r|s,a)\left[r + \gamma v_\pi(s')\right]$$

$$q_\pi(s,a) = E_\pi(G|s,a) = \sum_{s',r} P(s',r|s,a) E_\pi(G|s,a,s',r)$$

$$= \sum_{s',r} P(s',r|s,a)\left[r + \gamma E_\pi(G')\right] = \sum_{s',r} P(s',r|s,a)\left[r + \gamma v_\pi(s')\right]$$

Bellman 方程 $\quad v_\pi(s) = \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a)\left[r(s,a) + \gamma v_\pi(s')\right]$

当用确定 $\pi$, 则 $a$ 固定

$$v_\pi(s) = r(s,a) + \sum_{s'} \gamma P(s'|s,a) v_\pi(s')$$

矩阵形式 $v_\pi = r + \gamma P v_\pi$

$$v_\pi = (I - \gamma P)^{-1} r$$

Bellman 最优方程

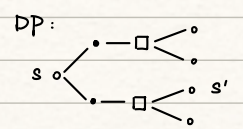
$$\pi^* = \arg\max_a q_*(s,a)$$

$$v_* = \max_a q_*$$

$$= \max_a E_\pi\left[r + \gamma v_*(s') \mid s, a\right]$$

$$= \max_a \sum_{s',r} (r + \gamma v_*(s')) P(s',r|s,a)$$

## 3. 动态规划 (DP)




用 Bellman 方程更新

值迭代时把 $\sum_a \pi(a|s)$ 换为 $\max_a$

根据 Bellman 方程 迭代求 $v_*(s)$ 和 $\pi^*$

策略迭代

策略估计 $\pi \to v$

$$v_{k+1}(s) = \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a)\left[r + \gamma v_k(s')\right]$$

策略改进 $v \to \pi$

$$\pi' = \arg\max_a q_\pi(s,a) = \arg\max_a r(s,a) + \sum_{s'} p(s'|s,a) v_\pi(s')$$

值迭代：忽略 $\pi$

$$v_{k+1}(s) = \max_a \sum_{s',r} p(s',r \mid s,a)\left[r + \gamma v_k(s')\right]$$

## 4. 强化学习

### 1) Monte Carlo 法

由实验，仿真学习，不需要 $p(s'|s,a)$ 和 $r(s,a)$

策略评价：已知 $\pi$ 求 $v_\pi(s)$

得到一组 $S_0, A_0, R_1 \cdots A_{N-1}, R_N$ 然后求 $G(s)$

用多次实验 $G(s)$ 的均值表示 $v_\pi(s)$

策略改进 / 控制：已知 $Q_\pi(s,a)$ 求 $\pi'$

在求均值时 $V(S_t) \leftarrow$ average( $S_t$ 的回报列表 )

也可记作

$$v(S_t) \leftarrow v(S_t) + \frac{1}{N}\left[G_t - v(S_t)\right]$$

非静态时，将 $\frac{1}{N}$ 换为 $\alpha$

```
Input : π
real number V(s) , for each s
empty list Returns(s) , for each s
Loop epoch :
    Experiment generate S0, A0, R1, S1, A1 ··· RT by π.
    G = 0
    for t = T-1, ···, 2, 1, 0
        G = γG + R_{t+1}
        if St doesn't exist in S0, ···, S_{t-1}
            append G to list Return(St)
            V(St) = average( Return(St))
        end
    end
end
```

### 2) 时间差分法 (TD)

Prediction : $v(S_t) \leftarrow v(S_t) + \alpha\left[R_{t+1} + \gamma v(S_t) - v(S_t)\right]$

Control : Sarsa $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\left[R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - v(S_t)\right]$

Q-learning $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\left[R_{t+1} + \gamma \max_a Q(S_t, a) - v(S_t)\right]$

MC :

o — • — □ - ··· - • — □

$S_t$ 可能在 $s_1 \cdots s_n$ 中取值

$G = \gamma G + R_t$ 添到 $S_t$ 的

TDO :

boostrap o → • → o

biased 有偏 $E[R_{t+1} + \gamma v(S_t)] \neq v(S_t)$

| | G list | V |
|---|---|---|
| $s_1$ | $G_1 \cdots \underline{G_1}$ | $v(s_1) = \text{Mean}(\vec{G}_1)$ |
| $s_2$ | $\underline{G_2}$ | $v(s_2)$ |
| ⋮ | ⋮ | ⋮ |
| $s_n$ | $G_n$ $\underline{G_n}$ | $v(s_n)$ |

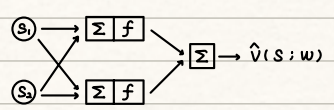
### 3) 逼近法 — 神经网络

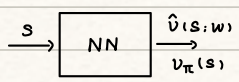
Prediction : 若已知 $v_\pi(s)$，正常训练

若未知 $v_\pi(s)$

SGD : Monte Carlo 边算 $G_t$ 边训练

Semi-Gradient : TDO 边算 $R_{t+1} + \gamma \hat{v}(S_{t+1}; w_t)$ 边训练




$$\hat{v}(s;w) = \beta_0 + \beta_1 \cdot \frac{1}{1+\exp(\alpha_{01} + \alpha_{11}S_1 + \alpha_{21}S_2)} + \beta_2 \cdot \frac{1}{1+\exp(\alpha_{02} + \alpha_{12}S_1 + \alpha_{22}S_2)}$$

$$w_{t+1} = w_t - \frac{1}{2}\alpha \nabla \overline{VE}(w_t)$$

$$\overline{VE}(w) = \sum \frac{1}{2}\left[v_\pi(s) - \hat{v}_\pi(s;w)\right]^2$$

Control : SGD 时很简单

Semi-Gradient 时 Sarsa，Q-learning 都能用

Bellman equation : $v(s)$ 和周围 $v(s')$ 的关系

$$v_\pi(s) = \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a)\left[r(s,a) + \gamma v_\pi(s')\right]$$

Bellman optimal equation

$$v_*(s) = \max_a \sum_{s',r} p(s',r|s,a)\left[r(s,a) + \gamma v_*(s')\right]$$

一. Model-based 需要 $p(s'|s,a)$ 和 $r(s,a)$ Ch5.

DP : 由 Bellman equation 更新 $v(s)$

Policy iteration $\begin{cases} \text{policy evaluation} & \pi \to v \\ \text{policy improvement} & v \to \pi \end{cases}$

Value iteration

二. Model-free RL

1. 表格法 Ch6.

1) Monte Carlo

$S_0, A_0, R_1 \cdots A_{N-1}, R_N$

从后往前 $G = \gamma G + R_t$, 收纳进 $S_t$ 的 list

$v_\pi(s)$ 取 s 所有 G 的均值

$$v^{(k+1)}(S_t) = v^{(k)}(S_t) + \frac{1}{N}\left[G_t - v^{(k)}(S_t)\right]$$

2) Temporal Difference : TD(0)

boostrap : 走一步看一步

$\circ \to \bullet \to \circ$

$S_0, A_0, R_1 \cdots A_{N-1}, R_N$

从后往前, 给 $v(S_t)$ 小步加上 $R_{t+1} + \gamma v(S_t) - v(S_t)$

$$v^{(k+1)}(S_t) = v^{(k)}(S_t) + \alpha\left[R_{t+1} + \gamma v^{(k)}(S_t) - v^{(k)}(S_t)\right]$$

Control $\begin{cases} \text{Sarsa} & \text{on-policy} \\ \text{Q-learning} & \text{off-policy} \end{cases}$

生成 $S_0, A_0 \cdots R_N$ 的 $\pi$

和被改进的 $\pi$ 不一样

2. 逼近法 : 神经网络 Ch7.

SGD : Monte Carlo

Semi-Gradient : TD0